

BALYASNY ASSET MANAGEMENT

Frontend Engineer Interview — Cheat Sheet

NEXT-ROUND PREP • Weekend Sprint Edition

React Internals

System Design

Performance

Machine Coding

Behavioural

Compiled from Glassdoor reports, hedge-fund FE interview research & role-specific intelligence • May 2026

CONFIRMED INTERVIEW STRUCTURE (from Glassdoor/WSO reports)

Round	Format	Duration	Weight
R1 (done)	HR/BD screening	30 min	Pass/Fail
R2 (NEXT)	Behavioural + System Design + 1-2 Coding	2× 45 min	HIGH
R3 (likely)	Deep CS/Architecture + stakeholder fit	45 min	CRITICAL

■ GLASSDOOR PATTERN

Interviewers want general knowledge depth + judgment, NOT speed-running LeetCode hard. Narrate your thought process.

① REACT INTERNALS & ARCHITECTURE

Why hooks work • Reconciliation • Concurrent rendering • Memoization

React Rendering Pipeline

■ **Concurrent Mode: Render phase is interruptible — Commit is synchronous & has no side-effect cancellation**



Fiber & Reconciliation — what they actually ask

Question	Answer to give
Why can't hooks be in conditionals?	Hooks are stored as a linked list on the fiber node — index order must be stable across renders
What is the "double-invoke" in StrictMode?	React calls render+cleanup twice in dev to expose side-effects in effects; no-op in prod
When does reconciler bail out?	Object.is() equality check on state/props; same element type; same key
What's the difference between Render & Commit?	Render = pure, interruptible (Concurrent), computes diff. Commit = synchronous, mutates DOM, runs effects
What triggers a full subtree re-render?	Context value reference change, parent re-render (without memo), key change

Memoization — correctness rules

useMemo	Memoize expensive computed values. Deps must be exhaustive. Never use for referential stability of simple objects — use useMemo only if computation is costly (>1ms).
useCallback	Memoize function identity. Only useful if passed as prop to memo() child or as dep to another hook. Cost: a closure allocation + dep comparison.
React.memo	Shallow prop comparison. Breaks on inline objects/functions unless also memoised upstream. Combine with useCallback for event handlers.
Selectors	Use Reselect/proxy-memoize in Redux; derive in Zustand with (state) => state.x. Selectors prevent unnecessary re-renders when unrelated state changes.

Context Performance Pitfalls

- **Problem:** Every consumer re-renders on ANY context value change, even if they only use one field.
- **Fix 1:** Split context into read/dispatch (DispatchContext + StateContext pattern).
- **Fix 2:** Use `useMemo` on the value object to stabilise reference: `const val = useMemo(() => ({price, qty}), [price, qty])`
- **Fix 3:** For HF use-cases — **prefer Zustand slices** or Redux selectors. Context is for low-frequency data (theme, auth).

Concurrent Rendering & Suspense

API	What it does	When to use
useTransition	Marks state update as low-priority; renders stay responsive	Search filters, tab switches
useDeferredValue	Defers a value update to background render	Debounce-like without delays
Suspense	Declarative loading boundary; integrates with data libs (React Query, Relay)	Code-split + async data
React.lazy	Route-level or component-level code splitting	Reduce initial bundle
startTransition	Wrap non-urgent updates; UI stays interactive	Expensive list re-renders

React Query / Cache Invalidation

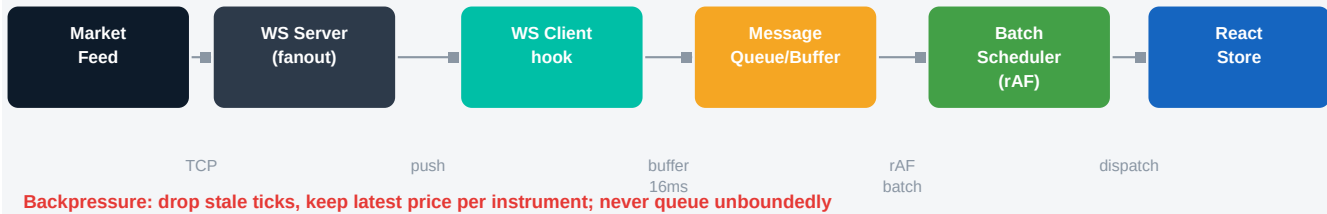
- **staleTime** — how long data is considered fresh (no background refetch). Set higher for reference data, 0 for live prices.
- **gcTime** (formerly cacheTime) — how long unused cache entries live. Independent of staleTime.
- **queryClient.invalidateQueries** — mark stale + triggers background refetch. Use after mutations.
- **Optimistic updates:** `onMutate` → `snapshot` → `rollback on error` — traders expect instant UI feedback.
- **For real-time:** disable React Query polling; use WebSocket subscription, update cache via `queryClient.setQueryData` .

② FRONTEND SYSTEM DESIGN

Real-time dashboards • Trading workstations • Risk monitors • Large grids

SCENARIO: Real-Time Trading Dashboard (5,000 price updates/sec)

WebSocket Architecture for 5k msgs/sec



WebSocket Layer: Single shared WS connection per session. Reconnect with exponential backoff + jitter. On reconnect, fetch snapshot then subscribe to delta stream.

Fanout / Message Bus: BroadcastChannel or SharedWorker for cross-tab sync. Single worker owns the WS; tabs subscribe. Prevents N connections to server.

Batching Strategy: Buffer incoming ticks in a Map. Flush via requestAnimationFrame (16ms) — only pushes latest price per instrument. Drops stale intermediate ticks.

State Architecture: Zustand slice per instrument. Selectors: usePrice(instrumentId). Only subscribing component re-renders. Redux RTK for order management (needs full history / time-travel).

Rendering: AG Grid with row/cell virtualisation. Pinned columns for key metrics. Delta row updates (not full data replace). CSS flash animations for price change (client-side only, no React state).

Backpressure: If queue > threshold, switch to "coarse mode" — only update every 100ms. Alert UI banner: "High load — reduced refresh rate". Never block the main thread.

Resilience: Partial failure: grey out stale cells with timestamp. Reconnect indicator in header. Heartbeat ping every 30s; treat missed pong as disconnect.

Observability: Track: messages/sec, render fps, time-to-first-byte, WS reconnect count. Expose to devtools via custom performance marks.

SCENARIO: Large-Scale Data Grid (100k+ rows)

Challenge	Solution	Tool
Rendering 100k rows	Windowed rendering — only render visible rows	AG Grid / react-window / react-virtual
Sorting/filtering perf	Web Worker for sort; return sorted indices only	Comlink + Worker Threads
Cell re-renders	Memoized cell renderers; AG Grid suppressRowTransform	React.memo + AG Grid config
Column pinning	Sticky positioning with transform; avoid layout thrash	CSS transform: translateX
Incremental data	Append rows via AG Grid transaction API (not full refresh)	ag.applyTransaction({add:[]})
Export large CSV	Stream Blob in Worker; avoid blocking main thread	Streams API + Worker

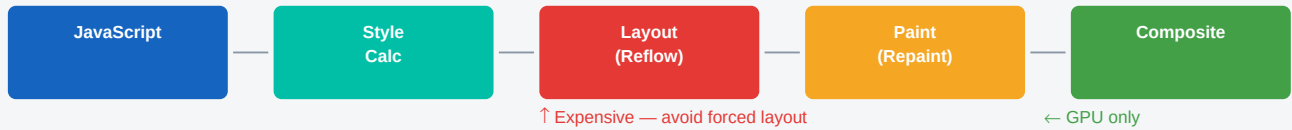
SCENARIO: Multi-Tab Synchronised Application

- **BroadcastChannel API:** Broadcast state mutations across tabs. `new BroadcastChannel('orders')` — `postMessage/onmessage`. No server needed.
- **SharedWorker:** Single JS context shared by all tabs. Owns WebSocket. Distributes messages. Falls back to BroadcastChannel if unsupported.
- **Local-first sync:** IndexedDB as source of truth. Sync to server async. Conflict resolution: last-write-wins or CRDT for collaborative scenarios.
- **Leader election:** Use `web-locks` API to elect one tab as "primary" (owns WS). Other tabs subscribe via BroadcastChannel.
- **Tab visibility:** `document.visibilityState` — pause WS updates when hidden; resume + snapshot on focus. Saves bandwidth.

③ FRONTEND PERFORMANCE ENGINEERING

The strongest signal for hedge fund frontend roles

Browser Rendering Pipeline



Repaint vs Reflow — What Triggers Them

Trigger	Type	Cost	Fix
offsetWidth, getBoundingClientRect	Forced Layout	■ CRITICAL	Batch reads before writes; cache values
width/height/margin change	Reflow + Repaint	■ HIGH	Use transform/opacity for animation
color/background change	Repaint only	■ MEDIUM	Promote to own layer: will-change: transform
transform/opacity change	Composite only	■ LOW	GPU-accelerated — preferred for animations
innerHTML	Full subtree reflow	■ CRITICAL	Use DOM APIs or React; never in hot path

Debugging Performance Degradation — Systematic Approach

- 1. Reproduce & measure → Chrome DevTools Performance tab. Record 5s. Note FPS drops.
- 2. Identify long tasks → Any task > 50ms is a "long task". Expand in flame chart.
- 3. React-specific → React DevTools Profiler. Check "Why did this render?". Look for unexpected renders.
- 4. JS bottlenecks → Bottom-up view in Performance tab. Sort by self-time. Find hot functions.
- 5. Memory → Memory tab. Heap snapshot. Compare before/after. Look for detached DOM nodes.
- 6. Network → WS frames tab. Message rate. Payload size. Compression (permessage-deflate).
- 7. Fix → Memoize, virtualise, batch, throttle, move to worker. Re-measure. Never guess.

Bundle Optimisation

- **Code splitting:** `React.lazy` + `Suspense` for routes. `dynamic import()` for heavy libs (charting, xlsx).
- **Tree shaking:** ESM only. Named imports from `lodash-es`, not `lodash`. Audit with `webpack-bundle-analyzer` / `vite-bundle-visualizer`.
- **Preload critical paths:** `link rel=preload` for fonts + main chunk. `modulepreload` for split points.
- **Caching:** Long-term cache via content-hashed filenames. `Cache-Control: max-age=31536000, immutable` for assets.
- **Worker offload:** Heavy computation (CSV parse, sort, charting data prep) → Web Worker. Keeps main thread for rendering.

Worker Threads Pattern

```
// Main thread const worker = new Worker(new URL("./sort.worker.ts", import.meta.url)); worker.postMessage({ data: rows, sortKey: "pn1" }); worker.onmessage = (e) => setState(e.data.sorted); // Worker (sort.worker.ts) self.onmessage = ({ data: { data, sortKey } }) => { const sorted = [...data].sort((a,b) => b[sortKey] - a[sortKey]); self.postMessage({ sorted }); };
```

④ STATE MANAGEMENT DEEP DIVE

Zustand vs Redux vs Context • Selectors • Normalisation

State Management Comparison

	Zustand	Redux Toolkit	Context API
Bundle size	~1kb	~15kb	0kb (built-in)
Re-render control	Selector slices	useSelector	All consumers
Best for	Local perf state	Complex global	Low-freq data
DevTools	Yes	Yes (excellent)	No
BAM verdict	✓ Price ticks	✓ Order book	✗ Avoid for HF

Zustand Pattern for Price Ticks (BAM-style)

```
// store.ts const useMarketStore = create()(set => ({ prices: {} as Record, updatePrice: (id: string, price: number) => set((s) => ({ prices: { ...s.prices, [id]: price } })), })); // Component – only re-renders when THIS instrument changes const price = useMarketStore((s) => s.prices[instrumentId]);
```

- **Key insight:** The selector `(s) => s.prices[instrumentId]` means only the component subscribed to that instrument re-renders. Others are unaffected.
- **Normalise state:** Never nest arrays. Use `Record` shape. Easier selectors, $O(1)$ lookup, surgical updates.

State Normalisation

BAM interviewers will ask about 5,000 msgs/sec. Un-normalised state forces array `find()` on every update $\rightarrow O(n)$. Normalised:

```
// ✗ Un-normalised (bad) state.orders = [{ id: "1", ... }, { id: "2", ... }] // Update: orders.find(o => o.id === id)  $\rightarrow O(n)$  every tick // ✓ Normalised (good) state.orders = { byId: { "1": {...}, "2": {...} }, allIds: ["1","2"] } // Update: state.orders.byId[id] = newOrder  $\rightarrow O(1)$ 
```

⑤ MACHINE CODING / PRACTICAL

Implement live • Narrate clearly • Production quality

Top Exercises Reported (BAM + similar HF roles)

curry() function: Confirmed from BAM Glassdoor Dec 2025 — implement variadic currying

EventEmitter: on/off/emit/once — classic; test with multiple handlers

Debounce / Throttle: From scratch, without lodash. Throttle = leading edge. Debounce = trailing.

Infinite Scroll: IntersectionObserver + async fetch + loading state + error boundary

Virtual List: Only render visible rows. Calculate offsets. Handle variable heights.

Typeahead/Autocomplete: Debounced fetch, keyboard nav, ARIA, cancel stale requests (AbortController)

Promise.all / race: Polyfill — tests understanding of Promise internals

curry() — Reference Implementation

```
function curry(fn) { return function curried(...args) { if (args.length >= fn.length) { return fn.apply(this, args); } return function(...args2) { return curried.apply(this, args.concat(args2)); }; }; } // const add = (a,b,c) => a+b+c; // curry(add)(1)(2)(3) === 6 ✓ // curry(add)(1,2)(3) === 6 ✓
```

Debounce — Reference Implementation

```
function debounce(fn, delay) { let timer; return function(...args) { clearTimeout(timer); timer = setTimeout(() => fn.apply(this, args), delay); }; } function throttle(fn, limit) { let inThrottle; return function(...args) { if (!inThrottle) { fn.apply(this, args); inThrottle = true; setTimeout(() => inThrottle = false, limit); } }; }
```

Virtual List — Core Logic

```
// Key formula const visibleCount = Math.ceil(containerHeight / itemHeight) + overscan; const startIndex = Math.floor(scrollTop / itemHeight); const endIndex = Math.min(startIndex + visibleCount, totalItems); const offsetY = startIndex * itemHeight; // translateY for container // Render only items[startIndex..endIndex] // Set outer container height = totalItems * itemHeight (prevents layout jump)
```

Communication Tips (interviewers care a LOT)

- Start with constraints: "Before I write code, let me clarify — do we need to handle edge cases X and Y?"
- Narrate intent before code: "I'll use a closure to capture the timer reference across invocations."
- Call out tradeoffs: "This is O(n) — in production I'd normalise the data structure."
- Test as you go: "Let me trace through this with add(1)(2)(3) mentally..."
- End with improvements: "Given more time I'd add: TypeScript generics, error boundaries, accessibility."

⑥ BEHAVIOURAL — "Can Traders Trust You?"

Ownership • Precision • Calmness • Accountability • Speed

Production incident?	STAR: What broke → Your immediate action → Root cause → Fix → Prevention. Emphasise: calmness, speed, measured impact (latency/\$ saved).
Disagreement with backend?	Show you raised it with data, proposed alternatives, then committed to the decision. Traders hate dithering.
Speed vs correctness?	"In trading, incorrect data is worse than slow data. I ship with feature flags, validate in staging with real-like data, then roll out incrementally."
Hardest performance issue?	Prepare a specific story: profiling tool used, what you found, what you changed, measured before/after.
Handling ambiguity?	"I clarify the most critical constraint first (latency? accuracy? availability?), build a thin vertical slice, validate, then iterate."
Why BAM / hedge fund FE?	"High-stakes UIs where a render lag has real P&L; consequences. I want my frontend decisions to matter to the business."

FinTech Framing — Your Advantage

- Always frame answers with **measurable outcomes**: latency reduction (ms), throughput (msgs/sec), error rate (%), revenue impact (£).
- Position FinTech experience as context for "I understand the cost of a wrong number or a slow render in this domain."
- Demonstrate domain knowledge: "I know bid/ask spread, order book depth, P&L; attribution — I can talk to traders directly."

7 JS FUNDAMENTALS AT EXPERT DEPTH

Event loop • Closures • Prototype chain • TypeScript

Event Loop — What BAM will probe

Queue	Examples	Priority
Call Stack	Sync code, function calls	1 — runs first
Microtask Queue	Promise.then, queueMicrotask, MutationObserver	2 — after each task, before paint
Macrotask Queue	setTimeout, setInterval, I/O, MessageChannel	3 — one per event loop turn
Animation Frame	requestAnimationFrame	Before next paint, after microtasks

Prototype Chain & Closure — Exam Questions

```
// Closure trap – classic interview question for (var i = 0; i < 3; i++) { setTimeout(() => console.log(i), 0); } // logs 3,3,3 – var is function-scoped } for (let i = 0; i < 3; i++) { setTimeout(() => console.log(i), 0); } // logs 0,1,2 – let is block-scoped } // Prototype function Animal(name) { this.name = name; } Animal.prototype.speak = function() { return this.name; }; // Dog.__proto__ === Animal.prototype // obj.__proto__.__proto__ === Object.prototype → null
```

TypeScript Patterns for HF Frontend

- **Discriminated unions:** Model order states — `type Order = { status: 'pending' } | { status: 'filled', price: number }`
- **Template literal types:** `type EventKey = `price.${string}`` — type-safe event names.
- **Branded types:** `type Price = number & { __brand: 'Price' }` — prevent mixing raw numbers with domain values.
- **Satisfies operator:** Validate shape without widening type — `const config = { ... } satisfies Config` .

⑧ WEEKEND SPRINT PLAN

Saturday + Sunday — 2-day prep schedule

Time Block	Focus	Output Goal
SAT Morning 9:00–12:00	React Internals deep dive Fiber, reconciler, Concurrent Mode, hooks internals	Can explain fiber lifecycle from memory
SAT Afternoon 13:00–16:00	WebSocket + State Architecture Zustand patterns, batching, normalisation, backpressure	Can whiteboard WS → React pipeline
SAT Evening 16:00–18:00	Machine Coding reps curry(), debounce(), EventEmitter, Promise.all	Implement all 4 without references
SAT Evening 19:00–20:00	Behavioural prep Prepare 4 STAR stories: incident, perf win, disagreement, ambiguity	4 polished answers, under 3 min each
SUN Morning 9:00–12:00	System Design: Trading Dashboard Whiteboard full architecture, all layers, edge cases	Can present 45-min design session solo
SUN Afternoon 13:00–15:30	Performance Deep Dive Browser pipeline, React profiler, bundle analysis, workers	Can answer "debug this slowdown" cold
SUN Afternoon 15:30–17:00	JS Fundamentals + TS Event loop, closures, prototype, discriminated unions	Ace any trivia-style rapid-fire round
SUN Evening 17:00–18:30	Mock Interview (solo) Time yourself: 1 system design + 1 code + 2 behavioural	Identify gaps, sleep confident

Priority Matrix — Where to Spend Time

Tier 1 — Must Know Cold	Tier 2 — Know Well	Tier 3 — Know Exists
Frontend System Design (trading dashboard, grid, multi-tab)	JS Fundamentals depth (event loop, prototype, closures)	Server Components tradeoffs
React Rendering Internals (Fiber, reconciler, concurrent)	TypeScript advanced patterns	CSS performance (layers, containment)
WebSocket Architecture (batching, backpressure, reconnect)	Machine Coding reps (curry, debounce, virtual list)	Testing strategy (RTL, Playwright)
Performance Debugging (profiler, flamegraph, reflow)	React Query / cache patterns	Accessibility (ARIA, WCAG)
State Management (Zustand, selectors, normalisation)	Behavioural STAR stories	CI/CD, monitoring

⑨ QUICK REFERENCE — THINGS TO MEMORISE

Flash-card style facts for rapid-fire rounds

React & JS	Performance & APIs
React.memo() : Shallow prop comparison; wrap stable components	rAF batching : ~16ms window; batch WS updates here for 60fps
useCallback(fn, []) : Stable fn reference; pair with memo() child	Long Task : >50ms on main thread; blocks rendering
useMemo(fn, []) : Memoize expensive value; not for referential stability	TTFB : Time To First Byte; server performance metric
Object.is() : React's equality check; NaN===NaN, +0!==-0	FCP : First Contentful Paint; first text/image visible
key prop : Change key = unmount + remount. Never use index in dynamic lists.	LCP : Largest Contentful Paint; < 2.5s is good
Strict Mode : Double-invokes render/effects in dev to expose bugs	CLS : Cumulative Layout Shift; < 0.1 is good
Fiber : Unit of work; linked list; enables time-slicing	INP : Interaction to Next Paint (replaced FID in 2024)
Reconciliation : Diff virtual DOM trees; same type = update, diff type = unmount+mount	will-change : Hint for GPU layer promotion; use sparingly
useLayoutEffect : Runs sync after DOM mutation, before paint. For measurements.	contain: strict : CSS containment; scopes layout/paint to element
useEffect : Runs async after paint. For subscriptions, fetches.	BroadcastChannel : Cross-tab messaging; same origin only
startTransition : Marks update as non-urgent; keeps UI responsive	SharedWorker : Single JS context shared across tabs; owns WS
Suspense boundary : Catches Promise throws from lazy/data libs; shows fallback	AbortController : Cancel fetch/async on component unmount

Key Resources — Bookmark These

Resource	Why
Frontend Interview Handbook	frontendinterviewhandbook.com — Algorithms + system design for FE
GreatFrontEnd	greatfrontend.com — Best FE system design resource; real trading/dashboard scenarios
React Docs — Performance	react.dev/learn/render-and-commit + useMemo/useCallback reference
web.dev Performance	web.dev/performance — Core Web Vitals, browser pipeline, rendering
AG Grid Docs	ag-grid.com/react-data-grid — Virtual rendering, transactions, pinned rows
Zustand GitHub	github.com/pmndrs/zustand — Selector patterns, middleware, devtools

■ **The meta-skill**: Hedge fund interviewers are evaluating whether traders can trust your UI. Every answer should signal: you understand the cost of latency, you measure before optimising, you own incidents, and you think in systems — not just components.